【扔物线 Compose 课程讲义】3.8-CompositionLocal - 是状态，但又不完全是

CompositionLocal 是什么

适用场景

用法

其他

关于讲义

讲义是对课程内容中**所有结论的集中汇总**。所以在看完每节课程之后，迅速来看一遍讲义，对课程内容做回顾，会有非常好的吸收效果。

面试前不要背讲义

在面试之前，通过快速浏览一遍讲义来确认所有知识都没有忘记，是个很好的备战方式。

但**不要用【背讲义上的知识点】的方式来备战**。最有效的浏览方式是：

浏览讲义中的所有内容，并：

- 看到「懂，并且记得背后的原因（或技术细节）」的，直接跳过；
- 看到「懂，但细节已经忘了」的，快速回看当节课程，把知识捡起来；
- 看到「已经不太懂 / 完全忘了」的，认真重看当节课程，把知识学回来。

疑问或意见

如果你发现讲义或课程有错误，或有别的什么意见，可以向任何一个你认识的助教反馈。

如果认可我们的课程，还请给个好评！ 评价链接：rengwuxian.com/cp

`CompositionLocal` 是什么

- 表面上看，是「针对 Composition 的、可以穿透函数的局部变量」；但其实作用更接近「不用显式传递的函数参数」。
- 它和函数参数的区别：
  - 函数参数是由下而上：函数负责规定参数的用途，调用者负责按规定来填写；
  - `CompositionLocal` 是由上而下：`CompositionLocal` 自身规定出自己的用途，具体的提供者和使用者各自按这份规定来提供和使用。

适用场景

- 上下文；
- 主题。

用法

- 创建：

Kotlin 复制代码

```kotlin
val name = compositionLocalOf { error("LocalName 没有提供值") }
val name = staticCompositionLocalOf { error("LocalName 没有提供值") }
```

  - 最好以 `local-` 或者 `Local-` 打头；
  - `compositionLocalOf()` 和 `staticCompositionLocalOf()` 的区别在于 `compositionLocalOf()` 会精准订阅、值改变时精准 Recompose，所以性能的消耗在于订阅阶段；而 `staticCompositionLocalOf()` 不订阅、值改变时全量 Recompose，所以性能的消耗在于更新阶段。因此 `compositionLocalOf()` 更时候「值可会改变」的场景，而 `staticCompositionLocalOf()` 适合「值不会改变或几乎不会改变」的场景。
- 提供：

Kotlin 复制代码

```kotlin
CompositionLocalProvider(LocalXxx provides xxx) {
    ...
}
```

> 可以用逗号 `,` 来分隔多个 `LocalXxx provides xxx` 。

- 使用：

Kotlin 复制代码

```kotlin
LocalXxx.current
```

> `CompositionLocal.current` 的 `get()` 是个 Composable 函数，因此它需要在 Composable 环境里使用。

其他

- 可以穿透多层函数；
- 可以嵌套多个 `CompositionLocalProvider()` 来提供同一个 `CompositionLocal` 使用，多层之间互不干扰；
- 可以设置默认值，但「没有有意义的默认」的时候不要强行设置，应该抛异常，更有利于开发调试。

扔物线学堂 - 秉心(1437182)